



Tribhuvan University
Institute of Science and Technology

Amrit Campus

Project Work Report on
Blockchain Based Land Registry

Under the Supervision of

Mr. Dharendra Kumar Yadav
Department of Computer Science & Information Technology
Amrit Campus, Thamel, Kathmandu

In partial fulfillment of the requirements for the Bachelor of Science in
Computer Science and Information Technology (B.Sc. CSIT) of Tribhuvan
University

Submitted by:

Mr. Abhishek Ojha / Roll No.79010148 / Batch 2079
Mr. Bivu Paudel / Roll No.79010176 / Batch 2079

Submitted to:

Department of Computer Science & Information Technology
Amrit Campus, Thamel, Kathmandu
Tribhuvan University

July, 2026



Tribhuvan University
Institute of Science and Technology
Amrit Campus
Department of Computer Science &
Information Technology

CERTIFICATE OF DECLARATION

We hereby declare that no part of this work has been previously submitted to the University or any other institution, either for evaluation or any other purpose. We affirm that the content of this work is the product of our own efforts and has not been derived from any other source except for those in references.

Declared by:

.....

Abhishek Ojha

Symbol No: 79010148/079

.....

Bivu Paudel

Symbol No: 79010176/079



Tribhuvan University
Institute of Science and Technology
Amrit Campus
Department of Computer Science &
Information Technology

Supervisors' Recommendation Letter

I hereby recommend that the project titled "**Blockchain Based Land Registry**" prepared by **Abhishek Ojha [79010148/079]**, **Bivu Paudel [79010176/079]** under my supervision, under my supervision, has been completed satisfactorily and is hereby submitted for evaluation.

.....

Mr. Dhirendra Kumar Yadav

Project Supervisor

Department of CSIT

Amrit Campus, TU

Date:.....



Tribhuvan University
Institute of Science and Technology
Amrit Campus
Department of Computer Science &
Information Technology

CERTIFICATE OF APPROVAL

This is to certify that the project report entitled “**Blockchain Based Land Registry**” was carried out by **Abhishek Ojha**[79010148/079] and **Bivu Paudel** [79010176/079] under the guidance and supervision of **Mr. Dharendra Kumar Yadav**. This report represents the candidate’s original work and has been completed in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Information Technology (B.Sc. CSIT) at Tribhuvan University

.....
Mr. Dharendra Kumar Yadav
Project Supervisor
Department of CSIT
Amrit Science Campus

.....
Mr. Dabbal Singh Mahara
Head / Coordinator
Department of CSIT
Amrit Science Campus

.....
External Examiner
Institute of Science & Technology
Tribhuvan University

.....
Internal Examiner
Institute of Science & Technology
Tribhuvan University

ACKNOWLEDGEMENT

The successful completion of this final year project titled “**Blockchain Based Land Registry System**” would not have been possible without the guidance, support, and encouragement of several individuals and institutions.

We would like to express our sincere gratitude to our supervisor, **Mr. Dharendra Kumar Yadav**, for his continuous guidance, technical insights, and valuable feedback throughout the development of this project. His suggestions and encouragement played a crucial role in shaping the direction and successful completion of the system.

We are also deeply thankful to our project coordinator, **Mr. Dabbal Singh Mahara**, for his constant support, coordination, and provision of necessary academic guidance during the entire project period.

We would like to extend our special thanks **to Mr. Akkal Bist** for his valuable ideas, suggestions, and consistent encouragement, which greatly contributed to improving the quality of this project.

We also express our gratitude to the concerned government employees and land administration staff who provided us with essential insights into real-world land record management processes. Their practical explanations helped us better understand the domain and design a more realistic system.

Finally, we would like to thank our friends, family members, and the faculty members of our institution for their continuous support, motivation, and encouragement throughout this academic journey.

Abhishek Ojha (79010148/079)

Bivu Paudel (79010176/079)

Date:03/07/2026

ABSTRACT

This project presents the design and implementation of a Land Record Management System that enhances the integrity, transparency, and traceability of land ownership records. Traditional land administration systems are often centralized and vulnerable to data manipulation, lack of transparency, and inefficient record tracking. To address these limitations, the proposed system introduces cryptographic integrity mechanisms including SHA-256 hashing, Hash Chain, and Merkle Tree structures to ensure tamper detection and verifiable record history.

The system is developed using a modular architecture with a Spring Boot backend, a React-based frontend, and PostgreSQL as the database management system. It supports role-based access for Super Admin, Malpot Officer, and Citizen, enabling secure workflows for land registration, ownership transfer, and record verification. Each land record and associated document is secured using cryptographic hashes, while audit logs ensure complete traceability of system activities.

The implementation demonstrates successful integration of frontend, backend, and database components along with efficient verification of data integrity through hash-based validation techniques. Although implemented as a proof of concept, the system validates the feasibility of applying blockchain-inspired techniques for improving land record management systems and provides a foundation for future enhancements in secure digital land administration.

Keywords: Land Record Management, Blockchain, Merkle Tree, Hash Chain, SHA-256, Data Integrity, Spring Boot, React, PostgreSQL.

Table of Contents

Contents

CERTIFICATE OF DECLARATION	i
Supervisors' Recommendation Letter	ii
CERTIFICATE OF APPROVAL	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
Table of Contents	vi
List of Abbreviations	ix
List of Figures	x
List of Tables	xi
Chapter 1: Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3. Objectives of the Project	2
1.4. Scope	3
In Scope	3
Out of Scope	3
1.5 Limitations:	4
Chapter 2: Background Study and Literature Review	5
2.1. Background Study	5
2.2 Literature Review	5
Chapter 3: System Analysis	7
3.1. Requirement Analysis	7
3.1.1 Functional Requirements	7
3.1.2 Non-functional Requirements	9
3.2 Feasibility Analysis	10
3.2.1 Technical Feasibility	10
3.2.2 Economic Feasibility	10
3.2.3 Operational Feasibility	10
3.2.4 Schedule	11

3.3 Object-Oriented Analysis Approach.....	11
3.3.1 Class Diagram.....	11
3.3.2 Database Schema.....	13
Chapter 4: System Design.....	14
4.1 System Architecture.....	14
4.2 System Modules.....	16
4.2.1 User Management Module.....	16
4.2.2 Land Record Management Module	16
4.2.3 Ownership Transfer Module.....	16
4.2.4 Integrity Verification Module.....	16
4.2.5 Audit Log Module	16
4.3 Process Modeling.....	17
4.3.1 Sequence Diagram	17
4.3.2 Object Diagram.....	18
4.3.3 State Diagram	19
4.3.4 Component diagram.....	20
4.3.5 Deployment Diagram.....	21
4.5 Algorithm Details.....	24
4.5.1 SHA-256 Hashing.....	24
4.5.2 Merkle Tree Algorithm.....	25
4.5.3 Integrity Verification Process	26
Chapter 5: Implementation	27
5.1 Software Development Life Cycle (SDLC).....	27
5.1.1 Requirement Analysis.....	28
5.1.2 System Design	28
5.1.3 Coding and Implementation.....	28
5.1.4 Testing	28
5.1.5 Maintenance and Documentation	29
5.2 Tools	29
5.2.1 IntelliJ IDEA.....	29
5.2.2 Visual Studio Code.....	29
5.2.3 Draw.io	29
5.2.4 Web Browser	30
5.3 Technologies Used.....	30
5.3.1 Java	30
5.3.2 Spring Boot.....	30
5.3.3 PostgreSQL.....	30
5.3.4 Flyway	31
5.3.5 JSON Web Token (JWT).....	31
5.3.6 React	31
5.3.7 TypeScript.....	31
5.3.8 Vite	31

5.3.9 Tailwind CSS.....	31
5.3.10 shadcn/ui	32
5.3.11 Zustand	32
5.3.12 SHA-256.....	32
5.3.13 Hash Chain.....	32
5.3.14 Merkle Tree	32
Chapter 6: Testing.....	33
6.1 Unit testing.....	33
6.2 Integration testing	34
6.3 System testing	35
Chapter 7: Conclusion and Future Recommendations	37
7.1 Conclusion:	37
7.2 Future scope:	37
8. References.....	38
9. Appendices.....	39
Appendix A: Screenshot of the Web Application.....	39
Appendix B: Screenshot of the Source Code.....	44
MerkleTreeEngine.java	44
HashChainVerifier.java	45
landRecordapi.ts	46
Log of Visits to Supervisor	47

List of Abbreviations

API	Application Programming Interface
DLT	Distributed Ledger Technology
DXA	Twentieth of a Point (measurement unit)
IPFS	InterPlanetary File System
JSON	JavaScript Object Notation
KYC	Know Your Customer
PoC	Proof of Concept
REST	Representational State Transfer
SHA	Secure Hash Algorithm
UI	User Interface

List of Figures

Figure 1 Use Case Diagram of the system.....	8
Figure 2 Work Schedule	11
Figure 3 Class Diagram	12
Figure 4 Database Schema.....	13
Figure 5 System Architecture	15
Figure 6 Sequence Diagram.....	17
Figure 7 Object Diagram	18
Figure 8 State diagram	19
Figure 9 Component Diagram	21
Figure 10 Deployment Diagram	23
Figure 11 SHA-256 Hash Generation.....	24
Figure 12 Merkle Tree Structure.....	25
Figure 13 Hash Verification.....	26
Figure 14 Software Development Lifecycle	27
Figure 15 Login Page.....	39
Figure 16 Registration Page.....	40
Figure 17 Citizen Interface	41
Figure 18 Admin Dashboard.....	41
Figure 19 Officer Dashboard	42
Figure 20 View Records	42
Figure 21 Verify Integrity	43
Figure 22 Source code of MerkleTree Engine.....	44
Figure 23 Source code of HashChain Verifier.....	45
Figure 24 SourceCode of LandRecord API	46

List of Tables

Table 1: Test cases for Unit Testing.....	34
Table 2: Test cases for Integration Testing	35
Table 3: Test cases for System Testing	36

Chapter 1: Introduction

1.1 Background

Land ownership is a fundamental component of economic and social systems worldwide. Traditional land record management systems primarily rely on paper-based documentation and centralized databases maintained by government authorities. These systems are often vulnerable to unauthorized modifications, data inconsistency, limited transparency, and single points of failure, making the verification and management of land ownership records challenging.

Blockchain technology has introduced concepts such as immutability, transparency, and cryptographic verification that have influenced the development of secure record management systems. Among these, cryptographic techniques such as SHA-256 hashing, Hash Chains, and Merkle Trees provide effective mechanisms for ensuring data integrity and detecting unauthorized modifications. These techniques enable efficient verification while maintaining the consistency and traceability of records.

This project implemented and evaluated a Proof of Concept (PoC) Land Record Management System that applies blockchain-inspired cryptographic techniques to secure digital land records. The system was developed using React, Spring Boot, and PostgreSQL, incorporating SHA-256 hashing, Hash Chain, and Merkle Tree structures to provide tamper detection and efficient record verification. The implementation demonstrated that these techniques can enhance the integrity, transparency, and reliability of land record management while providing a foundation for future research in secure digital land administration systems.

1.2 Problem Statement

Traditional land record management systems face several technical and operational challenges. Centralized storage of land records creates a single point of failure, making the system vulnerable to unauthorized modifications, data loss, and security breaches. Maintaining a complete and verifiable history of land ownership is also difficult, reducing transparency and complicating the verification of ownership records. Furthermore, conventional database systems permit authorized modifications to stored data, increasing the risk of record tampering and compromising data integrity. As a result, stakeholders must rely heavily on the managing authority to ensure the accuracy, authenticity, and consistency of land records.

To address these challenges, this project implements a Proof of Concept (PoC) Land Record Management System that incorporates blockchain-inspired cryptographic techniques, including SHA-256 hashing, Hash Chains, and Merkle Trees, to strengthen record integrity and traceability. The system aims to provide a secure mechanism for detecting unauthorized modifications, maintaining an auditable history of land records, and enabling efficient verification of data integrity while preserving the simplicity of a web-based land management application.

1.3. Objectives of the Project

The primary objectives of this PoC are:

- To demonstrate Immutability of the land records
- To establish Transparent Ownership History
- To implement Verifiable Ownership Transfers

1.4. Scope

In Scope

The scope of this project includes the development of a Proof of Concept (PoC) Land Record Management System with the following functionalities:

- User authentication and role-based access control for Super Admin, Malpot Officer, and Citizen users.
- Creation, updating, viewing, and management of land records within the system.
- Ownership transfer workflow involving citizen initiation, officer verification, and administrative approval.
- Generation of SHA-256 hashes for land records and uploaded documents to ensure data integrity.
- Implementation of Hash Chain and Merkle Tree structures for tamper detection and efficient integrity verification.
- Audit logging of critical system activities for traceability and accountability.
- Verification of record integrity using hash chain validation and Merkle proof generation.

Out of Scope

The following aspects are beyond the scope of this project:

- Integration with official government land registry systems or national identity databases.
- Legal validation of land ownership or compliance with land administration regulations.
- Financial operations such as online payments, land valuation, or taxation.
- Dispute resolution, arbitration, or handling of legal ownership conflicts.
- Deployment in a production environment or support for multi-jurisdictional land administration systems.

1.5 Limitations:

Some of the limitations are listed below:

- Internet connectivity is required to access the system.
- The system is developed as a Proof of Concept and is not intended for production deployment.
- The system does not provide legal validation of land ownership or dispute resolution mechanisms.
- The current implementation supports a limited number of users and records for demonstration purposes.

Chapter 2: Background Study and Literature Review

2.1. Background Study

Land record management is the process of recording, maintaining, and updating information related to land ownership, property boundaries, and ownership transfers. It plays a vital role in ensuring legal ownership, reducing land disputes, facilitating property transactions, and supporting effective land administration. Traditionally, land records have been managed through paper-based documentation and centralized digital databases maintained by government authorities. Although these approaches have improved record accessibility, they remain vulnerable to unauthorized modifications, limited transparency, single points of failure, and difficulties in verifying the integrity of historical records.

Blockchain is a distributed ledger technology that provides data integrity, transparency, and tamper resistance through cryptographic techniques. While a complete blockchain system relies on decentralized consensus mechanisms, several of its underlying data structures, such as cryptographic hash functions, Hash Chains, and Merkle Trees, can be applied independently to enhance the security of digital records. A cryptographic hash function generates a unique fixed-length hash for each record, enabling the detection of any unauthorized modification. Hash Chains link records sequentially, while Merkle Trees organize record hashes into a hierarchical structure that enables efficient integrity verification.

2.2 Literature Review

Blockchain technology has been widely explored by researchers as a means of improving the security, transparency, and integrity of land record management systems. Several studies have proposed blockchain-based frameworks that utilize distributed ledgers, smart contracts, and cryptographic techniques to prevent unauthorized modifications, eliminate single points of failure, and maintain an immutable history of land ownership. Other researchers have investigated the use of cryptographic data structures, including Hash Chains and Merkle Trees, to provide efficient integrity verification and tamper detection while reducing storage and computational overhead.

The effectiveness of these approaches has also been demonstrated through practical implementations. The National Agency of Public Registry (NAPR) in Georgia collaborated with BitFury to secure more than 1.5 million land titles using blockchain technology. Sweden's Lantmäteriet implemented a pilot project with ChromaWay to streamline property transactions, while pilot initiatives in India explored blockchain for secure land record management. Similarly, the Dubai Land Department and the Bitland project in Ghana demonstrated the application of blockchain for transparent and tamper-resistant land registration. These studies indicate that blockchain and its underlying cryptographic techniques have significant potential to improve land administration. Motivated by these findings, this project implements a Proof of Concept that applies SHA-256 hashing, Hash Chains, and Merkle Trees to strengthen the integrity, traceability, and verification of digital land records.

Chapter 3: System Analysis

3.1. Requirement Analysis

Requirement analysis is the process of identifying, analyzing, and documenting the functional and non-functional requirements of a system. It provides a clear understanding of the expected behavior of the system and serves as the foundation for system design and implementation. The requirement analysis for the Land Record Management System was conducted to identify the necessary functionalities, user interactions, and performance expectations required to meet the project objectives.

3.1.1 Functional Requirements

Functional requirements describe the specific services and operations that the system must perform to satisfy user requirements. They define the interactions between users and the system and specify the core functionalities implemented in the application.

The functional requirements of the system are illustrated using the use case diagram shown below.

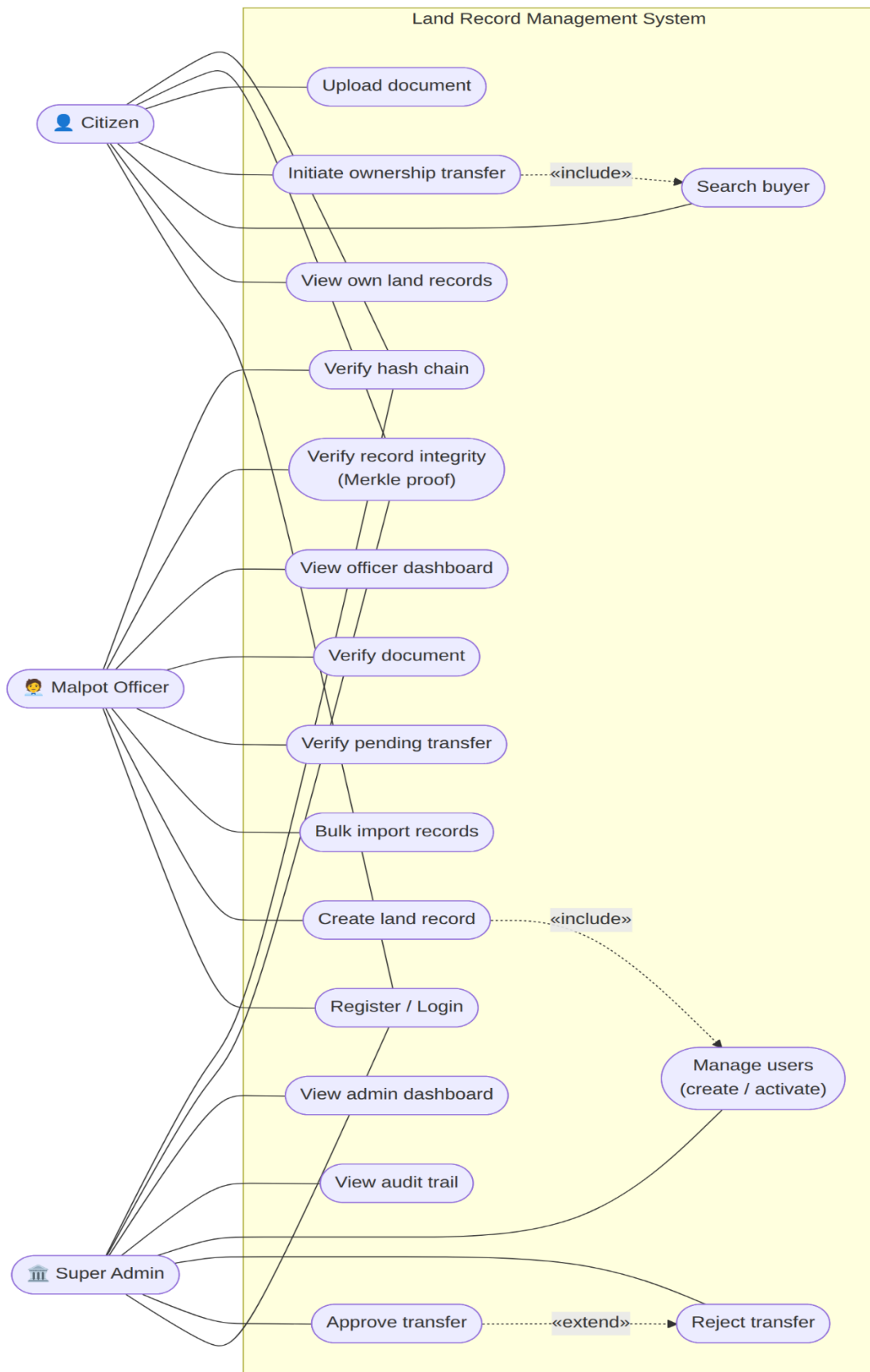


Figure 1 Use Case Diagram of the system

The major functional requirements of the system include:

- User registration and authentication.
- Role-based access control for Super Admin, Malpot Officer, and Citizen.
- Land record creation, updating, viewing, and deletion.
- Upload and management of land-related documents.
- Ownership transfer request and approval workflow.
- SHA-256 hash generation for land records and documents.
- Merkle Tree proof generation and record integrity verification.
- Hash Chain validation for sequential record verification.
- Audit logging of critical system activities.

3.1.2 Non-functional Requirements

Non-functional requirements define the quality attributes and performance characteristics of the system. They describe how the system operates rather than the specific functions it performs.

The non-functional requirements of the system are as follows:

- **Security:** The system uses JWT-based authentication, role-based authorization, and cryptographic hashing to ensure secure access and data protection.
- **Integrity:** SHA-256 hashing, Hash Chains, and Merkle Trees are implemented to detect unauthorized modifications and maintain record integrity.
- **Performance:** The application provides efficient record retrieval and integrity verification with acceptable response times.
- **Reliability:** Audit logs and cryptographic verification mechanisms ensure consistent and trustworthy system operation.
- **Usability:** The user interface is designed to be simple, intuitive, and easy to navigate for all user roles.
- **Maintainability:** The modular architecture supports future enhancements and simplifies maintenance.
- **Scalability:** The system can be extended to accommodate additional users, records, and functionalities.

3.2 Feasibility Analysis

Feasibility analysis is the process of evaluating whether a system can be successfully developed and implemented using the available resources, technologies, budget, and time. It examines various aspects of the project to determine its practicality and identifies whether the system can effectively meet its intended objectives. The feasibility analysis conducted for this project confirmed that the Land Record Management System was practical to develop within the available academic resources and timeline.

3.2.1 Technical Feasibility

Technical feasibility assesses whether the required technologies and development tools are available to implement the system. The system was developed using Spring Boot for the backend, React with TypeScript for the frontend, and PostgreSQL for database management. Security and data integrity were achieved through JWT-based authentication, SHA-256 hashing, Hash Chain, and Merkle Tree algorithms, which enable tamper detection and efficient record verification. All selected technologies are open source, well documented, and compatible with one another, making the implementation technically feasible.

3.2.2 Economic Feasibility

Economic feasibility evaluates the financial viability of the project. The system was developed entirely using open-source software, frameworks, and development tools, eliminating licensing costs. Existing computing resources were sufficient for development, testing, and deployment, resulting in minimal implementation expenses. Therefore, the project was economically feasible within the constraints of an academic environment.

3.2.3 Operational Feasibility

Operational feasibility determines whether the system can be effectively used within its intended environment. The developed system supports three user roles—Super Admin, Malpot Officer, and Citizen—with clearly defined responsibilities for managing land records and ownership transfers. The application provides a user-friendly interface, secure authentication, document management, and integrity verification features, enabling users to perform required operations efficiently within a controlled environment.

3.2.4 Schedule

Schedule feasibility evaluates whether the project can be completed within the allocated time. The project was completed according to the planned schedule, including requirement analysis, system design, implementation, testing, documentation, and final evaluation. Appropriate planning and the use of established development frameworks ensured the successful completion of all project objectives within the available academic duration. The schedule for the project is given below:

GANTT CHART: 12 WEEKS												
Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Planning												
Environment Setup												
Smart Contract Development												
Backend Development												
Smart Contract Testnet Deployment												
Frontend Development												
Integration and Testing												
System Optimization and Refinement												

Figure 2 Work Schedule

3.3 Object-Oriented Analysis Approach

The system follows an object-oriented analysis approach where the system is modeled using classes, objects, and their interactions. Each major entity in the system such as users, land records, ownership transfers, and documents is represented as a class with defined attributes and methods. The relationships between these entities are represented using class diagrams, while the database schema is derived from the object model to ensure consistency between application logic and data storage.

3.3.1 Class Diagram

A Class Diagram is a static structural diagram used in object-oriented modeling to represent the classes of a system, their attributes, methods, and relationships. It provides a clear visualization of how different components of the system interact at a logical level.

In the proposed system, the class diagram represents core entities such as User, LandRecord, OwnershipTransfer, Document, and AuditLog. It defines the relationships among these classes, including associations between users and land records, transfer requests, and system-generated audit activities. This diagram helps in understanding the overall structure of the system and forms the basis for implementation of backend modules.

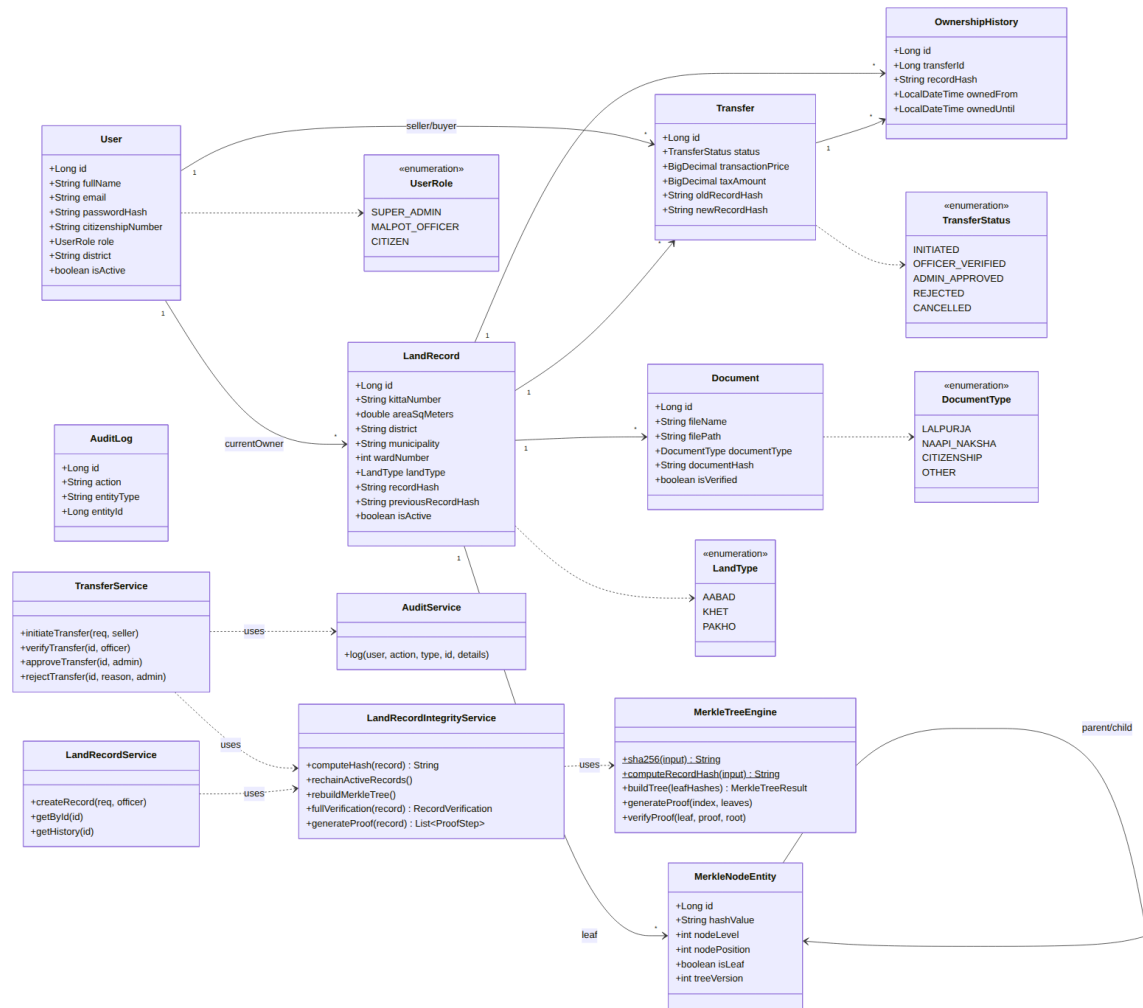


Figure 3 Class Diagram

3.3.2 Database Schema

The schema of the system was designed to ensure data consistency, eliminate redundancy, and maintain referential integrity among related tables.

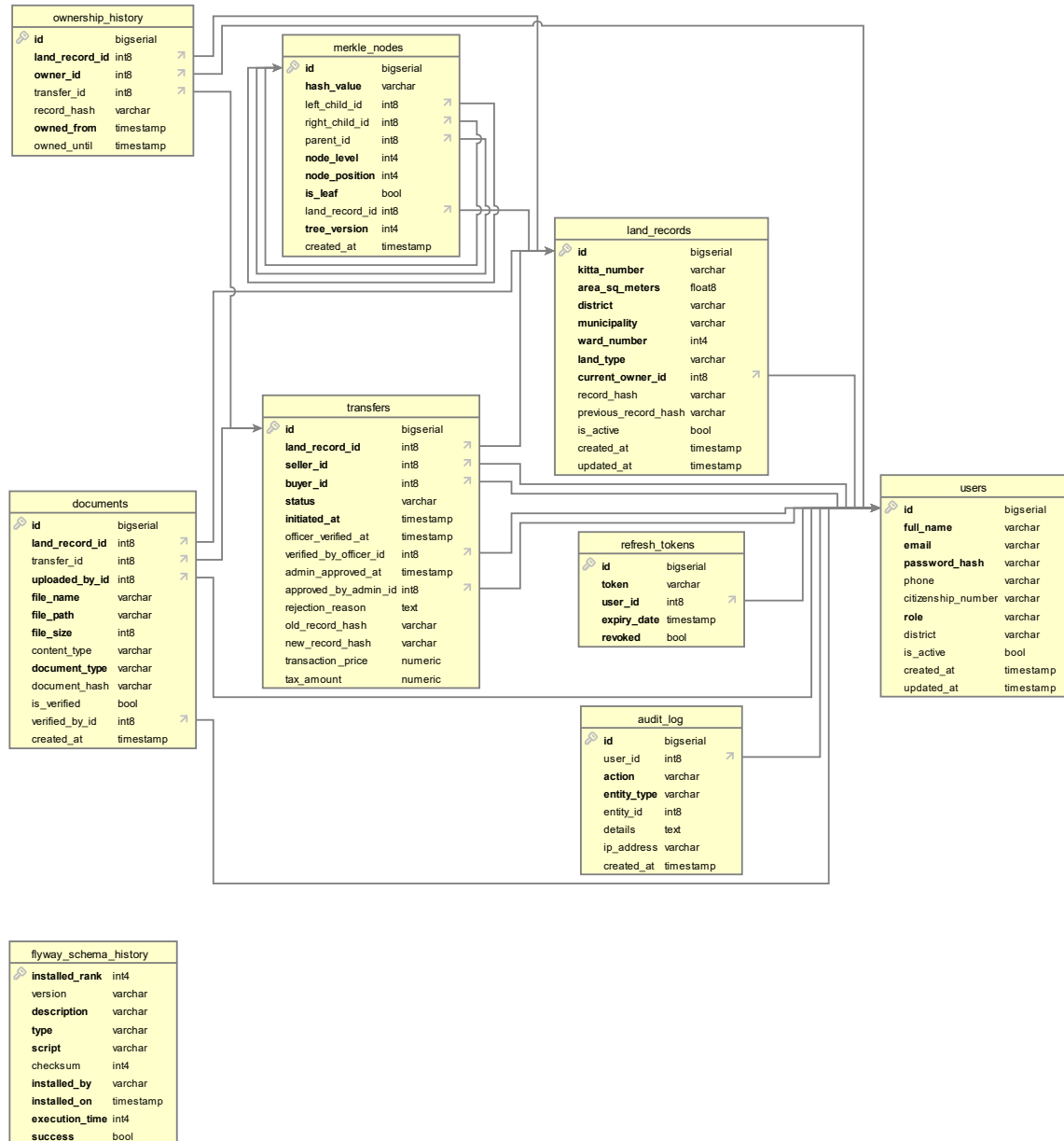


Figure 4 Database Schema

Chapter 4: System Design

System design is the process of defining the architecture, components, modules, database structure, and interactions of a system based on the requirements identified during system analysis. It serves as a blueprint for the implementation of the system by illustrating how different components work together to achieve the desired functionality. The Land Record Management System follows an object-oriented design approach with a modular architecture that ensures maintainability, scalability, and ease of integration.

4.1 System Architecture

The system follows a three-tier architecture consisting of the presentation layer, application layer, and data layer. The presentation layer is developed using React and provides interfaces for Super Admin, Malpot Officer, and Citizen. The application layer is implemented using Spring Boot and exposes RESTful APIs for handling business logic, authentication, land record management, ownership transfer, document management, and integrity verification. The data layer uses PostgreSQL for persistent storage, while SHA-256 hashing, Hash Chain, and Merkle Tree structures ensure data integrity and tamper detection.

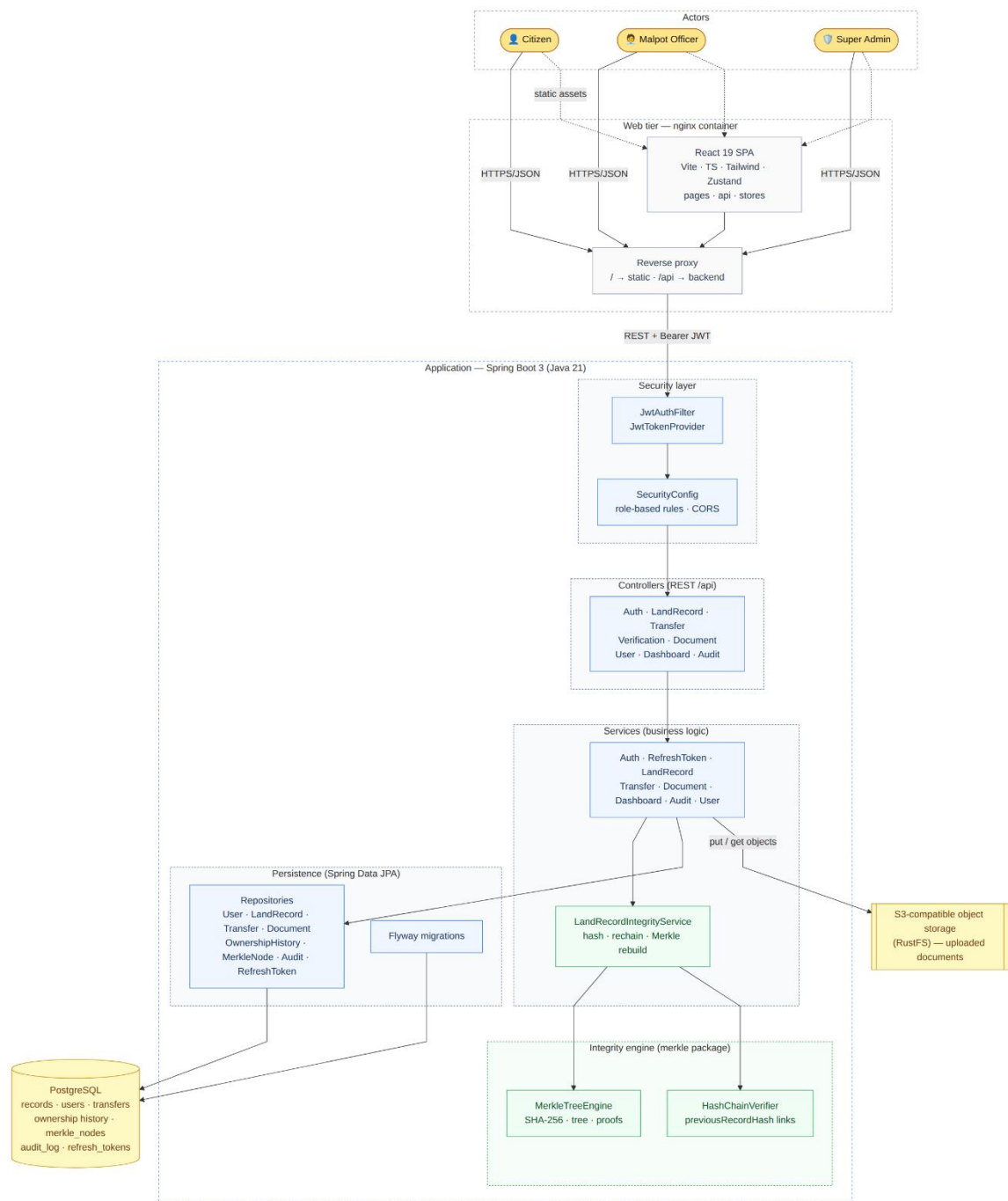


Figure 5 System Architecture

4.2 System Modules

The system is divided into several functional modules to ensure modularity and maintainability.

4.2.1 User Management Module

This module manages user registration, authentication, profile management, and role-based authorization. It supports three user roles: Super Admin, Malpot Officer, and Citizen, ensuring that users can access only the functionalities assigned to their respective roles.

4.2.2 Land Record Management Module

This module allows authorized users to create, update, view, and manage land records. It maintains ownership details, property information, and supporting documents while preserving data consistency throughout the system.

4.2.3 Ownership Transfer Module

The ownership transfer module manages the complete ownership transfer workflow. Citizens initiate transfer requests, Malpot Officers verify the submitted information, and the Super Admin performs the final approval or rejection of the transfer request.

4.2.4 Integrity Verification Module

This module ensures the integrity of land records using SHA-256 hashing, Hash Chain, and Merkle Tree structures. It generates cryptographic hashes, verifies record consistency, detects unauthorized modifications, and provides Merkle proof verification.

4.2.5 Audit Log Module

The audit log module records critical system activities, including land registration, ownership transfers, document updates, and administrative actions. These logs provide traceability and improve accountability within the system.

.

4.3 Process Modeling

Process modeling illustrates the sequence of activities and interactions performed by the system to accomplish specific tasks. It provides a graphical representation of the workflow, enabling a better understanding of how data flows through the system and how different users interact with various system components.

4.3.1 Sequence Diagram

In this system, sequence diagrams are used to illustrate processes such as land record registration, ownership transfer, and record verification. For example, during land record registration, the user submits data through the frontend, which is processed by the backend services and stored in the database, followed by a response confirming successful completion.

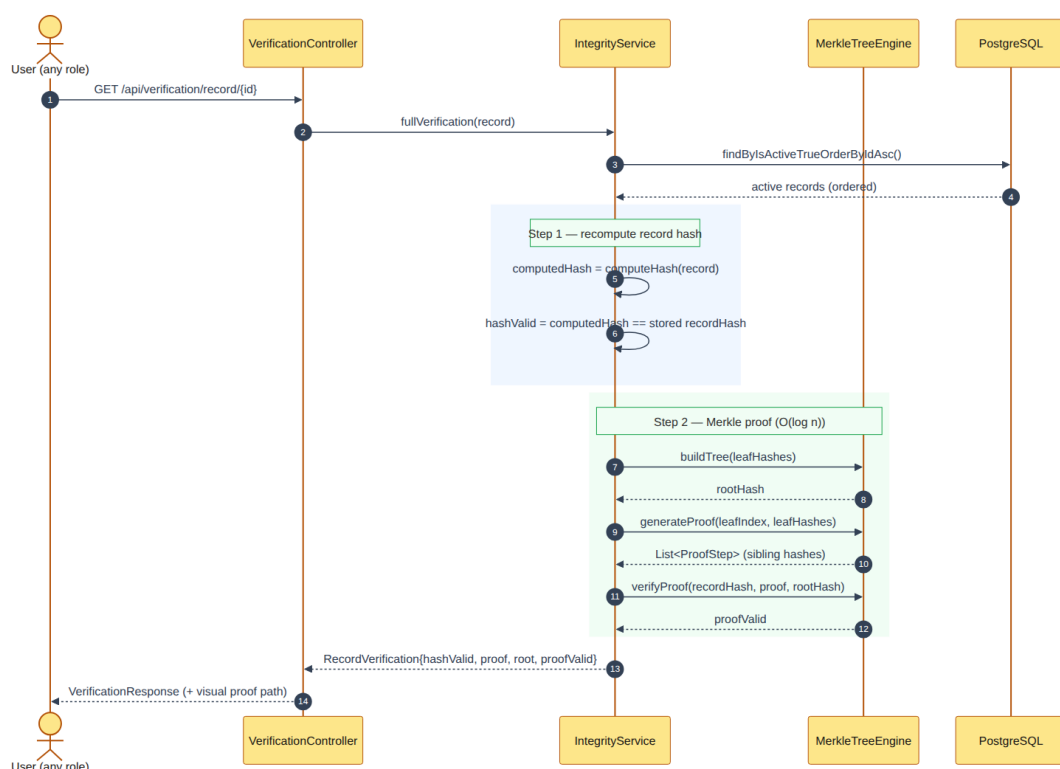


Figure 6 Sequence Diagram

4.3.2 Object Diagram

An Object Diagram represents a snapshot of the system at a specific point in time, showing instances of classes and their relationships. It provides a real-time view of how objects interact during system execution. In this system, the object diagram illustrates instances such as a specific user, a land record, and an ownership transfer request, along with their current state and relationships. It helps in understanding how system objects exist and are connected during runtime.

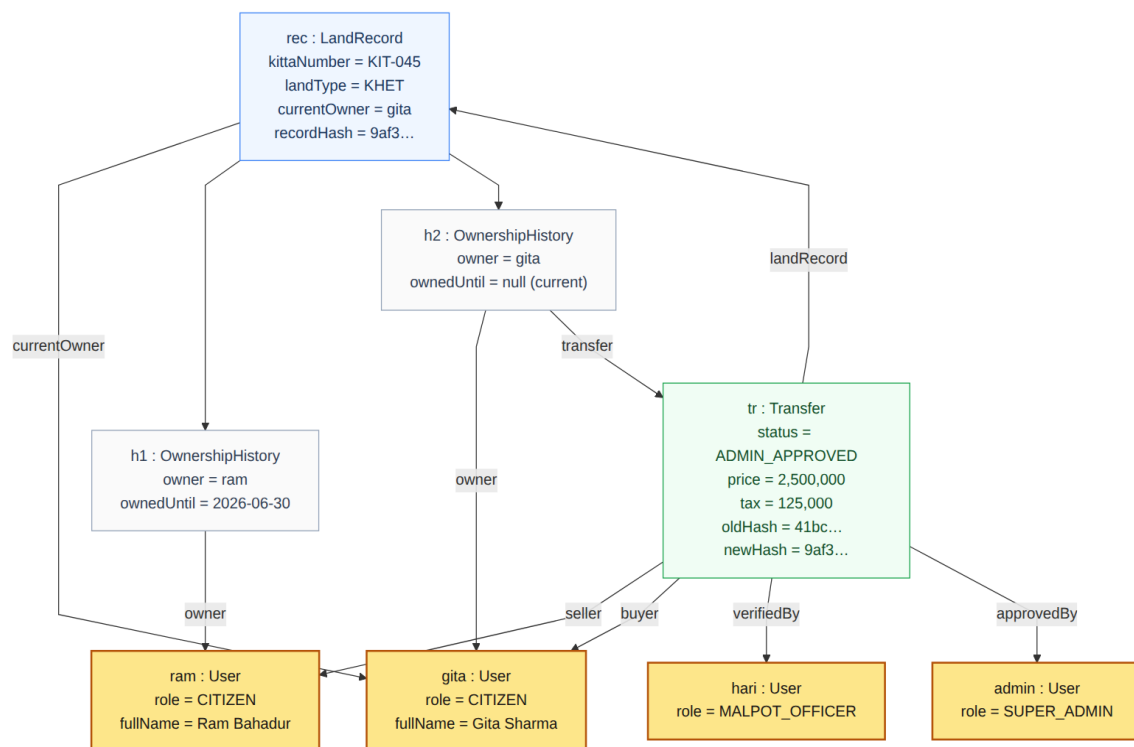


Figure 7 Object Diagram

4.3.3 State Diagram

A State Diagram represents the lifecycle of an object and shows how it transitions from one state to another based on system events. In this system, it is used to represent the lifecycle of entities such as land records and ownership transfer requests. For example, a land record may transition through states such as Created, Under Verification, Approved, and Transferred depending on user actions and system processes.

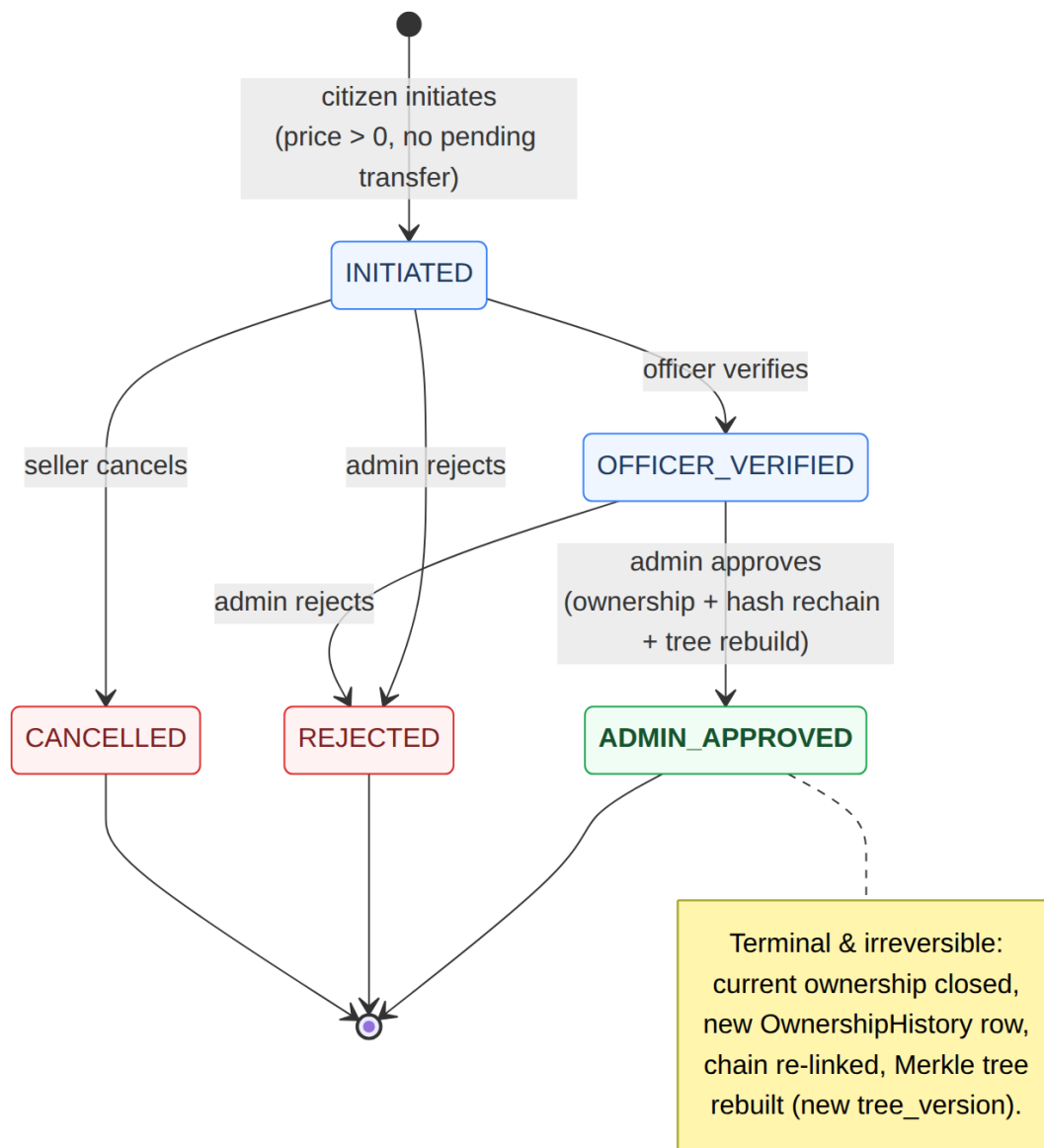


Figure 8 State diagram

4.3.4 Component diagram

In the Land Record Management System, the Component Diagram represents the interaction between the frontend application, backend services, database layer, and core functional modules. The frontend component handles user interaction and communicates with backend REST APIs. The backend is divided into modules such as user management, land record management, ownership transfer, and integrity verification. These components interact with the PostgreSQL database for persistent storage and ensure separation of concerns within the system architecture.

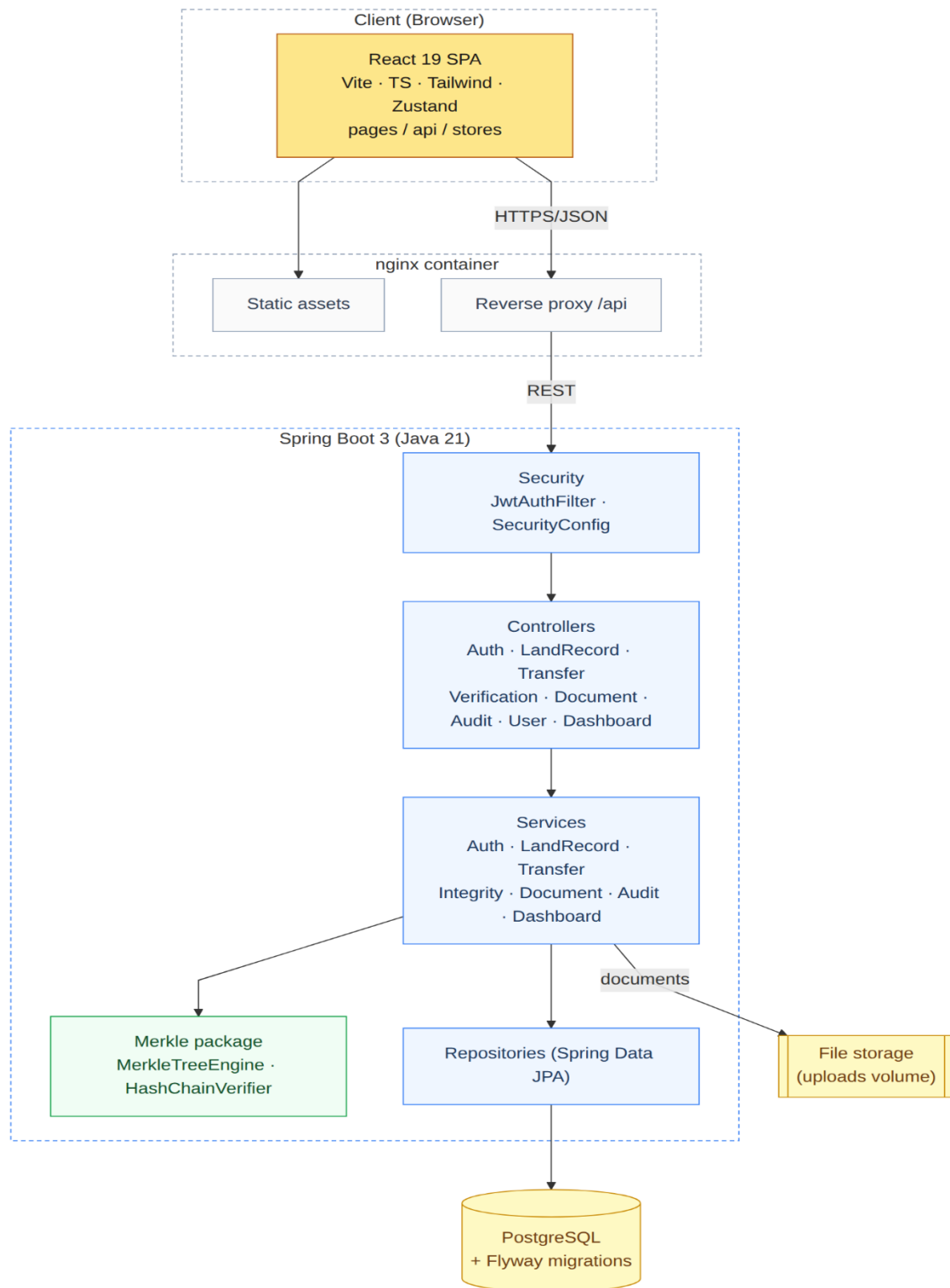


Figure 9 Component Diagram

4.3.5 Deployment Diagram

In the system, the Deployment Diagram illustrates the deployment of the web application in a client-server architecture. The client node represents the user's web browser, which

interacts with the frontend application. The application server hosts the Spring Boot backend, which processes business logic and handles API requests. The database server hosts the PostgreSQL database, which stores all system data including users, land records, transactions, and audit logs. Communication between these nodes is carried out through secure HTTP requests, ensuring proper separation between presentation, application, and data layers.

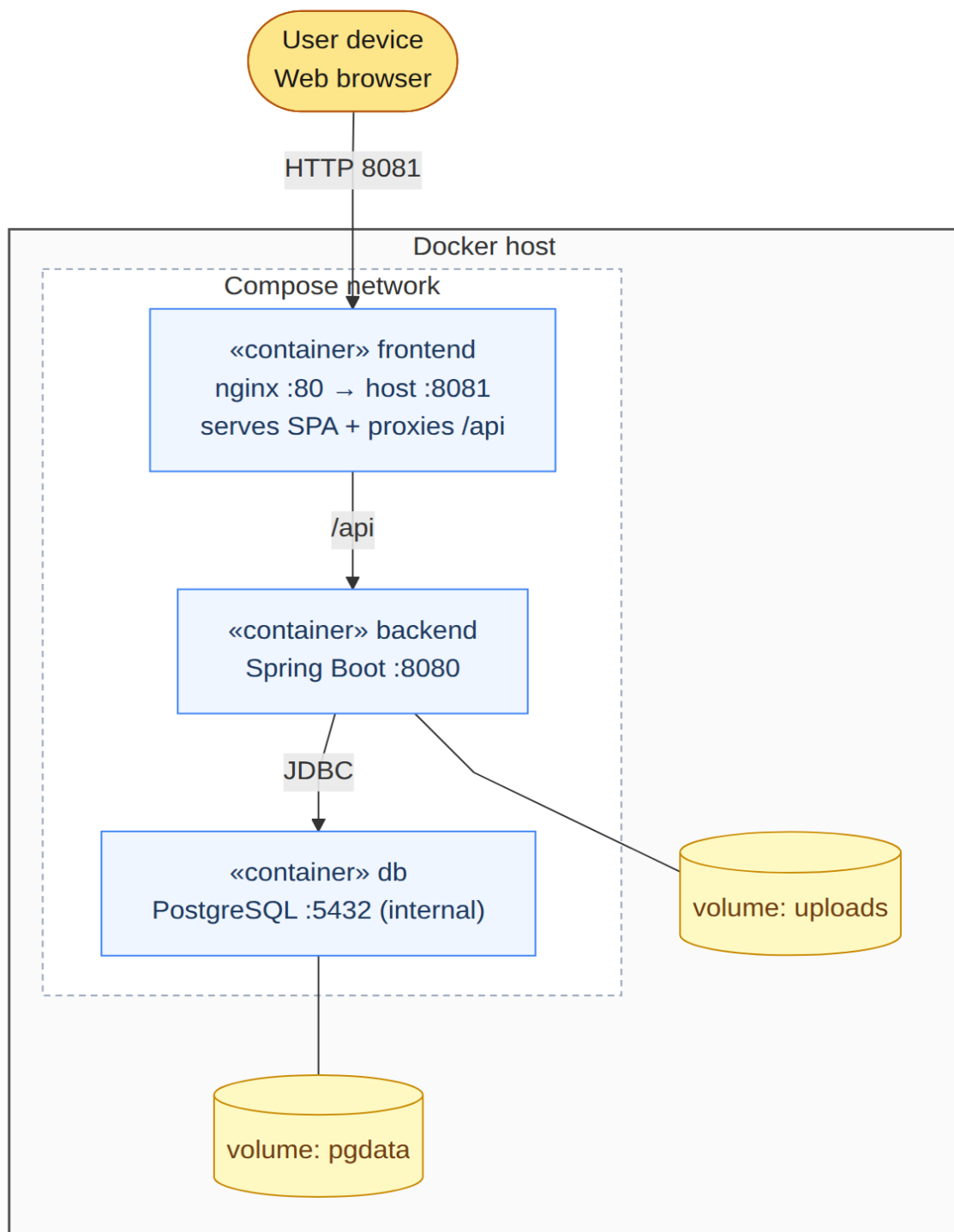


Figure 10 Deployment Diagram

4.5 Algorithm Details

This section describes the algorithms and cryptographic techniques implemented in the Land Record Management System. These algorithms ensure the integrity, consistency, and verifiability of land records by detecting unauthorized modifications and enabling efficient validation of stored data.

4.5.1 SHA-256 Hashing

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function that generates a fixed-length 256-bit hash value from input data of arbitrary length. In the system, SHA-256 is used to generate unique hash values for land records and uploaded documents. These hashes act as digital fingerprints, allowing the system to detect any unauthorized changes to stored data.

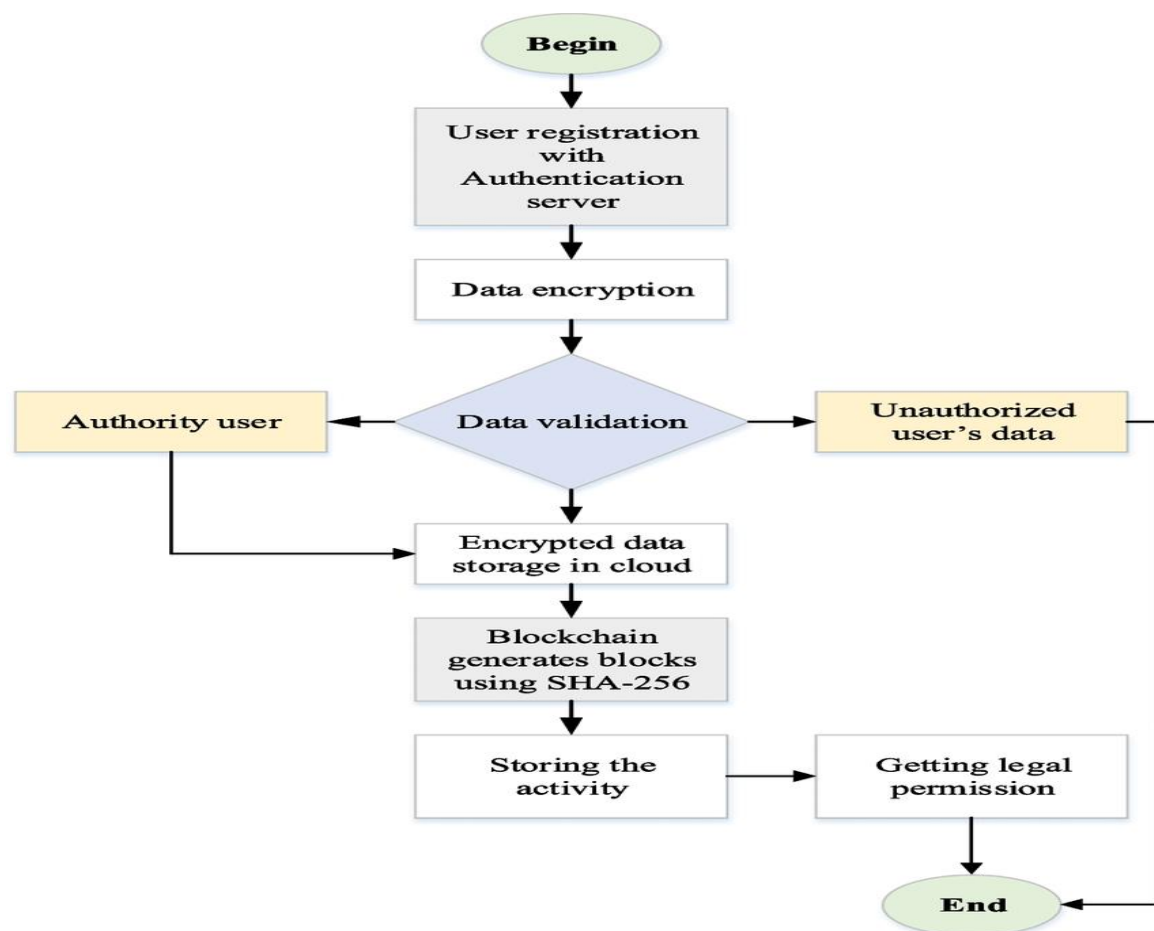


Figure 11 SHA-256 Hash Generation

4.5.2 Merkle Tree Algorithm

A Merkle Tree is a hierarchical data structure in which leaf nodes represent the hash values of individual records, while each parent node is generated by hashing the concatenation of its child nodes. The final hash, known as the Merkle Root, uniquely represents the integrity of the entire dataset. If any record is modified, the corresponding leaf hash and all subsequent parent hashes change, resulting in a different Merkle Root. In the system, the Merkle Tree is used to organize land record hashes and provide efficient integrity verification through Merkle proofs.

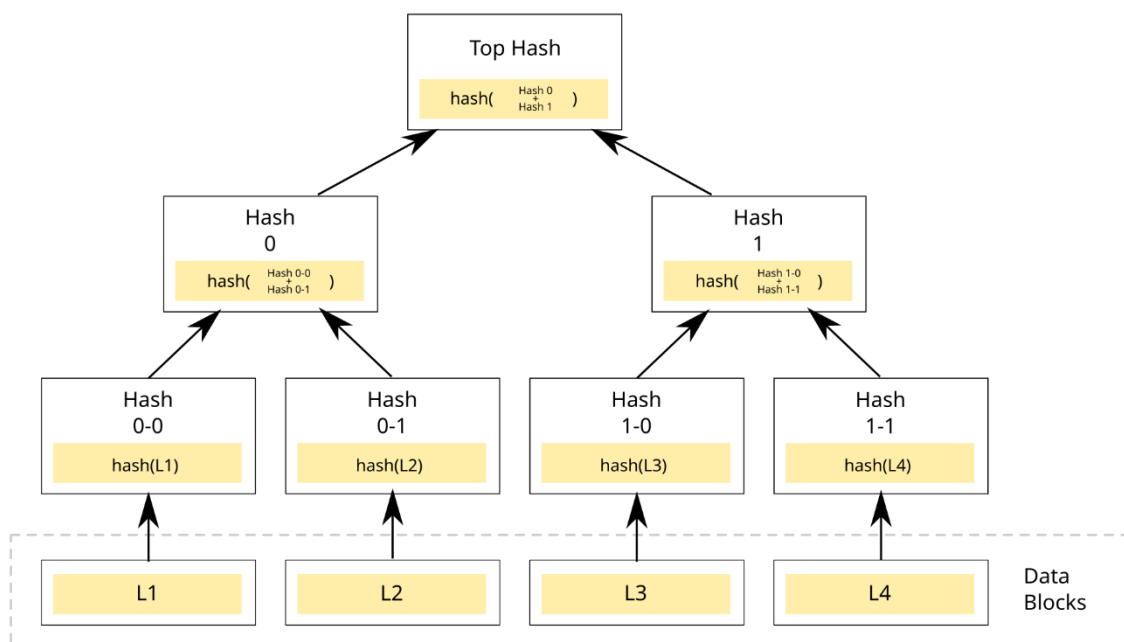


Figure 12 Merkle Tree Structure

4.5.3 Integrity Verification Process

The integrity verification process confirms that stored land records remain unaltered after their creation. During verification, the system recalculates the SHA-256 hash of the selected record and compares it with the previously stored hash. The corresponding Merkle proof is then generated and validated against the Merkle Root to verify the authenticity of the record. If the computed values match the stored values, the record is considered valid; otherwise, the system reports that the integrity of the record has been compromised.

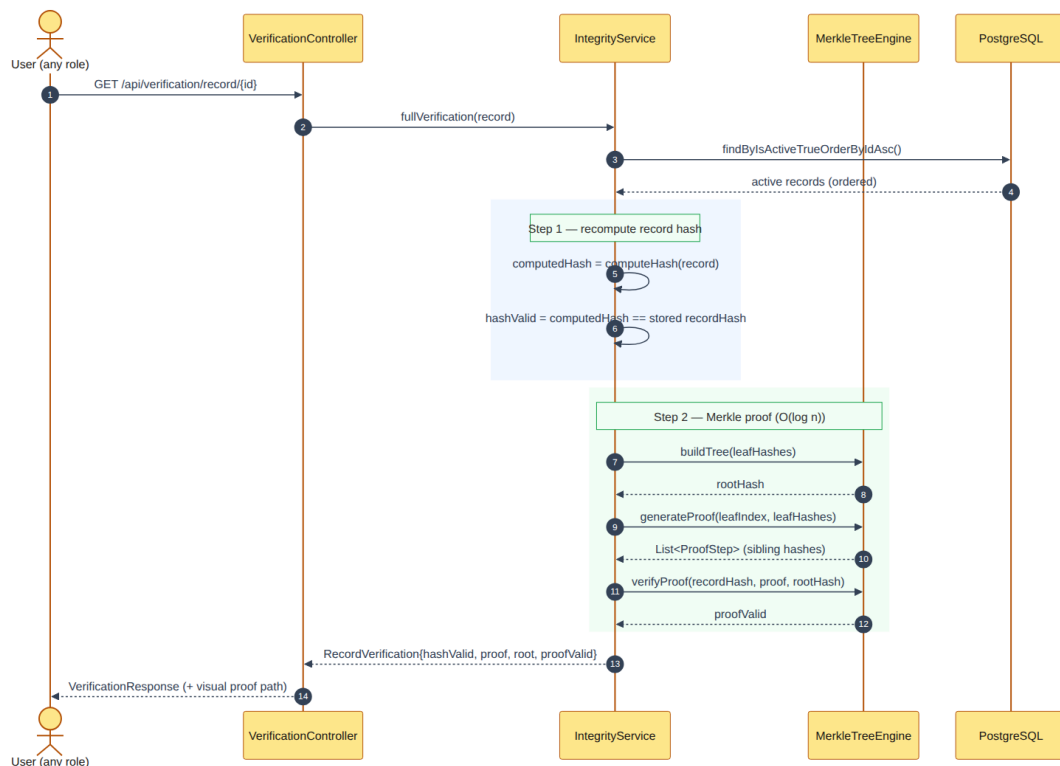


Figure 13 Hash Verification

Chapter 5: Implementation

Implementation is the process of transforming the proposed system design into a functional software application through coding, integration, testing, and deployment. It involves developing the required system modules, implementing the database, integrating the frontend and backend components, and verifying that the system satisfies the specified requirements. The Land Record Management System was developed following the Software Development Life Cycle (SDLC), ensuring a systematic approach throughout the development process.

5.1 Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured process used to develop software in a systematic and organized manner. It consists of several phases that guide the project from requirement gathering to deployment and maintenance. The major phases followed during the development of the system are described below.

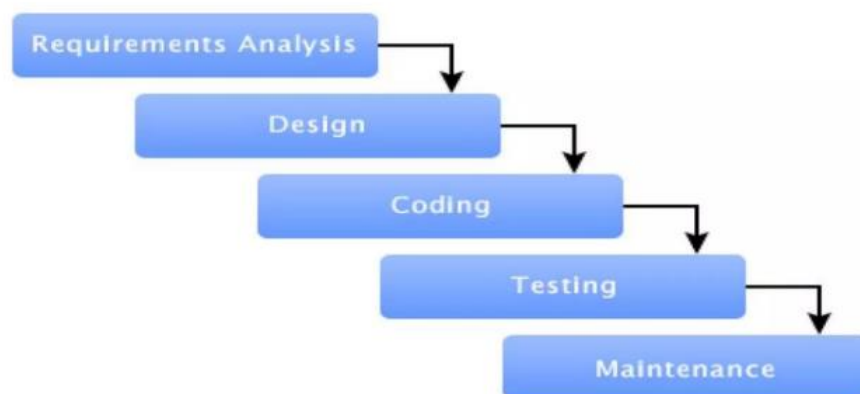


Figure 14 Software Development Lifecycle

5.1.1 Requirement Analysis

During this phase, the functional and non-functional requirements of the system were identified through analysis of existing land record management practices and project objectives. The required user roles, system functionalities, security requirements, integrity verification mechanisms, and ownership transfer workflow were defined to establish the scope of the project.

5.1.2 System Design

Based on the identified requirements, the overall system architecture and database structure were designed. User interfaces were planned for the Super Admin, Malpot Officer, and Citizen roles. The database schema, REST API structure, and system modules, including user management, land record management, document management, ownership transfer, audit logging, and integrity verification, were designed prior to implementation.

5.1.3 Coding and Implementation

The system was implemented using Spring Boot for backend development, React with TypeScript for the frontend, and PostgreSQL for database management. Individual modules were developed and integrated through RESTful APIs. Authentication and authorization were implemented using JWT, while SHA-256 hashing, Hash Chain, and Merkle Tree algorithms were integrated to provide tamper detection and efficient record integrity verification.

5.1.4 Testing

After implementation, the developed modules were tested individually and as an integrated system to verify their correctness and reliability. Unit testing was performed for major components and cryptographic algorithms, while system testing validated the complete workflow of authentication, land record management, ownership transfer, document management, and integrity verification. The testing methodology and results are presented in Section 5.2.

5.1.5 Maintenance and Documentation

Although the project was developed as an academic proof of concept, the source code was organized using a modular architecture to facilitate future maintenance and enhancement. Comprehensive documentation, including system design, API documentation, database design, and implementation details, was prepared to support future development, deployment, and maintenance of the system.

5.2 Tools

Software development tools were used throughout the project to support system design, implementation, testing, and documentation. These tools facilitated efficient development, debugging, visualization, and system verification.

5.2.1 IntelliJ IDEA

IntelliJ IDEA was used as the primary integrated development environment (IDE) for backend development. It provided features such as intelligent code completion, debugging, dependency management, and seamless integration with Spring Boot, enabling efficient implementation of the backend components.

5.2.2 Visual Studio Code

Visual Studio Code (VS Code) was used for frontend development. It provided a lightweight development environment with support for TypeScript, React, and various extensions that simplified coding, debugging, and project management.

5.2.3 Draw.io

Draw.io was used to design and document the system architecture, UML diagrams, flowcharts, Entity Relationship (ER) diagram, and database schema. It provided a convenient platform for creating clear and organized system diagrams used throughout the project documentation.

5.2.4 Web Browser

A modern web browser was used to access, test, and verify the functionality of the web application during development. It was also used to inspect the user interface, validate REST API responses, and ensure compatibility and responsiveness across different system features.

5.3 Technologies Used

The Land Record Management System was developed using a combination of modern programming languages, frameworks, databases, and cryptographic technologies. Each technology was selected based on its reliability, performance, and suitability for developing secure web-based applications.

5.3.1 Java

Java 21 was used as the primary programming language for backend development. It provides object-oriented programming features, platform independence, strong security, and extensive library support, making it suitable for developing scalable and maintainable enterprise applications.

5.3.2 Spring Boot

Spring Boot 3.x was used as the backend framework to develop the RESTful web services and implement the business logic of the application. It simplifies application development through dependency injection, auto-configuration, and embedded server support, reducing development time while improving maintainability.

5.3.3 PostgreSQL

PostgreSQL was used as the relational database management system to store user information, land records, ownership transfer requests, uploaded documents, and audit logs. It provides reliable data storage, supports complex queries, and ensures data consistency through transactional processing.

5.3.4 Flyway

Flyway was used to manage database versioning and schema migration. It enables controlled updates to the database structure by maintaining migration scripts, ensuring that the database remains synchronized with application development.

5.3.5 JSON Web Token (JWT)

JSON Web Token (JWT) was implemented to provide secure authentication and authorization. After successful login, a token is generated and used to authenticate subsequent requests, ensuring that only authorized users can access protected system resources.

5.3.6 React

React 19 was used to develop the frontend of the application. It is a component-based JavaScript library that enables the development of dynamic, responsive, and reusable user interfaces, improving both user experience and application maintainability.

5.3.7 TypeScript

TypeScript was used alongside React to provide static type checking and improved code reliability. It enhances JavaScript by detecting type-related errors during development, making large-scale applications easier to develop and maintain.

5.3.8 Vite

Vite served as the frontend build tool for the project. It provides a fast development environment with instant module replacement and optimized production builds, improving the overall development workflow.

5.3.9 Tailwind CSS

Tailwind CSS v4 was used for designing the user interface. It is a utility-first CSS framework that enables rapid development of responsive and consistent user interfaces while reducing the need for custom CSS code.

5.3.10 shadcn/ui

shadcn/ui was used as the user interface component library. It provides reusable, customizable, and accessible UI components, allowing the development of a consistent and modern user interface.

5.3.11 Zustand

Zustand was used for frontend state management. It provides a lightweight and efficient mechanism for sharing and managing application state across different React components, simplifying data flow within the application.

5.3.12 SHA-256

SHA-256 is a cryptographic hash function used to generate a unique 256-bit hash value for land records and uploaded documents. It enables the system to detect unauthorized modifications by producing a completely different hash value whenever the original data changes.

5.3.13 Hash Chain

The Hash Chain algorithm links records sequentially by incorporating the hash value of the previous record into the current record. This mechanism ensures that any modification to a record invalidates the integrity of subsequent records, providing tamper-evident storage.

5.3.14 Merkle Tree

The Merkle Tree is a hierarchical cryptographic data structure used to organize record hashes efficiently. Individual record hashes are combined recursively to generate a single Merkle Root, allowing rapid verification of data integrity without processing every record in the dataset.

Chapter 6: Testing

6.1 Unit testing

Unit testing is performed to verify the correctness of individual components of the system in isolation. The objective is to ensure that each core function works as expected before integrating it with other modules. In this system, unit testing focuses on authentication logic, land record creation, document handling, and cryptographic operations such as SHA-256 hashing, Hash Chain generation, and Merkle Tree verification.

The tests were executed at the backend level using JUnit-based test cases in the Spring Boot environment. Each module was tested independently by providing controlled input data and verifying the corresponding output.

Test Case ID	Test Scenario	Test Steps	Input Test Data	Expected Result	Actual Result	Pass/Fail
UT-01	User Login	Enter valid credentials and submit login form	Email: <u>ram@example.com</u> , Password: citizen123	User should be authenticated and redirected to dashboard	User successfully logged in and redirected	Pass
UT-02	User Registration	Create a new citizen account	Name, Email, Password	Account should be created successfully	Account created successfully	Pass
UT-03	Create Land Record	Officer creates a new land record	Owner details, land info	Land record should be stored with SHA-256 hash	Record created with hash generated	Pass
UT-04	SHA-256 Hash Generation	Verify hash	Land record data	Unique hash should be generated	Hash generated correctly	Pass

		creation for record				
UT-05	Document Upload	Upload land-related document	PDF/Image file	File should be stored with integrity hash	File uploaded and hash stored	Pass

Table 1 Test cases for Unit Testing

6.2 Integration testing

Integration testing is performed to verify the interaction between different modules of the system. It ensures that components such as frontend, backend, and database work together correctly as a unified system.

In this project, integration testing was carried out by executing full workflows involving multiple system modules, including user authentication, land record management, ownership transfer, and integrity verification. API communication between React frontend and Spring Boot backend was tested using RESTful requests, while database integration was verified using PostgreSQL queries.

Test Case ID	Test Scenario	Test Steps	Input Test Data	Expected Result	Actual Result	Pass/Fail
IT-01	Login to Dashboard Integration	Login and access dashboard	Valid user credentials	User should access role-based dashboard	Dashboard loaded correctly	Pass
IT-02	Land Record Storage Flow	Create land record via frontend	Land details	Data should be stored in DB with hash	Stored successfully in DB	Pass

IT-03	Ownership Transfer Flow	Citizen initiates transfer → officer verifies → admin approves	Transfer request data	Transfer should complete successfully	Workflow completed successfully	Pass
IT-04	Backend–Database Integration	API call to fetch records	GET request	Data should be retrieved correctly	Data retrieved from DB	Pass
IT-05	Document Verification Integration	Upload and verify document integrity	File upload	File hash should match stored hash	Verification successful	Pass

Table 2 Test cases for Integration Testing

6.3 System testing

System testing is conducted on the complete integrated system to verify that the application meets all functional and non-functional requirements. It ensures that the system behaves correctly in real-world scenarios and supports all user roles and workflows.

In this system, testing was performed by executing end-to-end workflows such as land registration, ownership transfer, and integrity verification. Special focus was given to verifying security features such as role-based access control, data integrity using SHA-256 hashing, Hash Chain validation, and Merkle Tree proof verification.

Test Case ID	Test Scenario	Test Steps	Input Test Data	Expected Result	Actual Result	Pass/Fail
ST-01	End-to-End User Workflow	Login → create record →	All role users	Complete workflow should	Workflow completed successfully	Pass

		transfer → verify		execute successfully		
ST-02	Role-Based Access Control	Access restricted features using wrong role	Citizen accessing admin panel	Access should be denied	Access correctly denied	Pass
ST-03	Integrity Verification	Modify data and verify hash chain	Altered record	System should detect tampering	Tampering detected successfully	Pass
ST-04	Merkle Tree Verification	Verify land record using Merkle proof	Record ID	Proof validation should succeed	Verification successful	Pass
ST-05	Audit Logging System	Perform system operations and check logs	Any CRUD operations	All actions should be logged	Logs recorded correctly	Pass

Table 3 Test cases for System Testing

Chapter 7: Conclusion and Future Recommendations

7.1 Conclusion:

This project successfully implemented a Proof of Concept (PoC) Land Record Management System that demonstrates the use of blockchain-inspired cryptographic techniques for secure and reliable digital land record management. The system provides role-based access control, land record management, ownership transfer, document management, audit logging, and data integrity verification using SHA-256 hashing, Hash Chains, and Merkle Trees.

The implementation and testing confirmed that the system meets its intended functional requirements while ensuring the integrity, transparency, and traceability of land records. Although developed for academic purposes and limited in scope, the project demonstrates the potential of cryptographic techniques to enhance conventional land record management systems and provides a foundation for future improvements and real-world applications.

7.2 Future scope:

The developed Land Record Management System can be further enhanced by adding additional features to improve its usability and functionality. One major improvement is the integration of Geographic Information System (GIS) technology to enable map-based visualization of land parcels and boundaries, making record management more intuitive.

The system can also be extended with notification features such as email or SMS alerts to inform users about ownership transfers, approvals, and document verification updates. This would improve communication and transparency within the system.

Additionally, advanced search and reporting capabilities can be introduced to support efficient retrieval and analysis of land records for administrative use. The system may also be integrated with national digital identity and government platforms to enhance security and real-world applicability.

8. References

- [1] IJARIIE, "Blockchain-based land records system using Hyperledger Fabric," International Journal of All Research Education & Scientific Methods, 2023.

- [2] Springer Publishing, "BLRS: An automated land records management system using blockchain," 2023.

- [3] Rifan Alitica Journal, "Smart land record system using blockchain," 2023.

- [4] IJSRED, "Decentralised record management system in land data using blockchain," International Journal of Scientific Research and Education, 2023.

- [5] Codearies, "Blockchain-based digital land registry platforms," 2024.

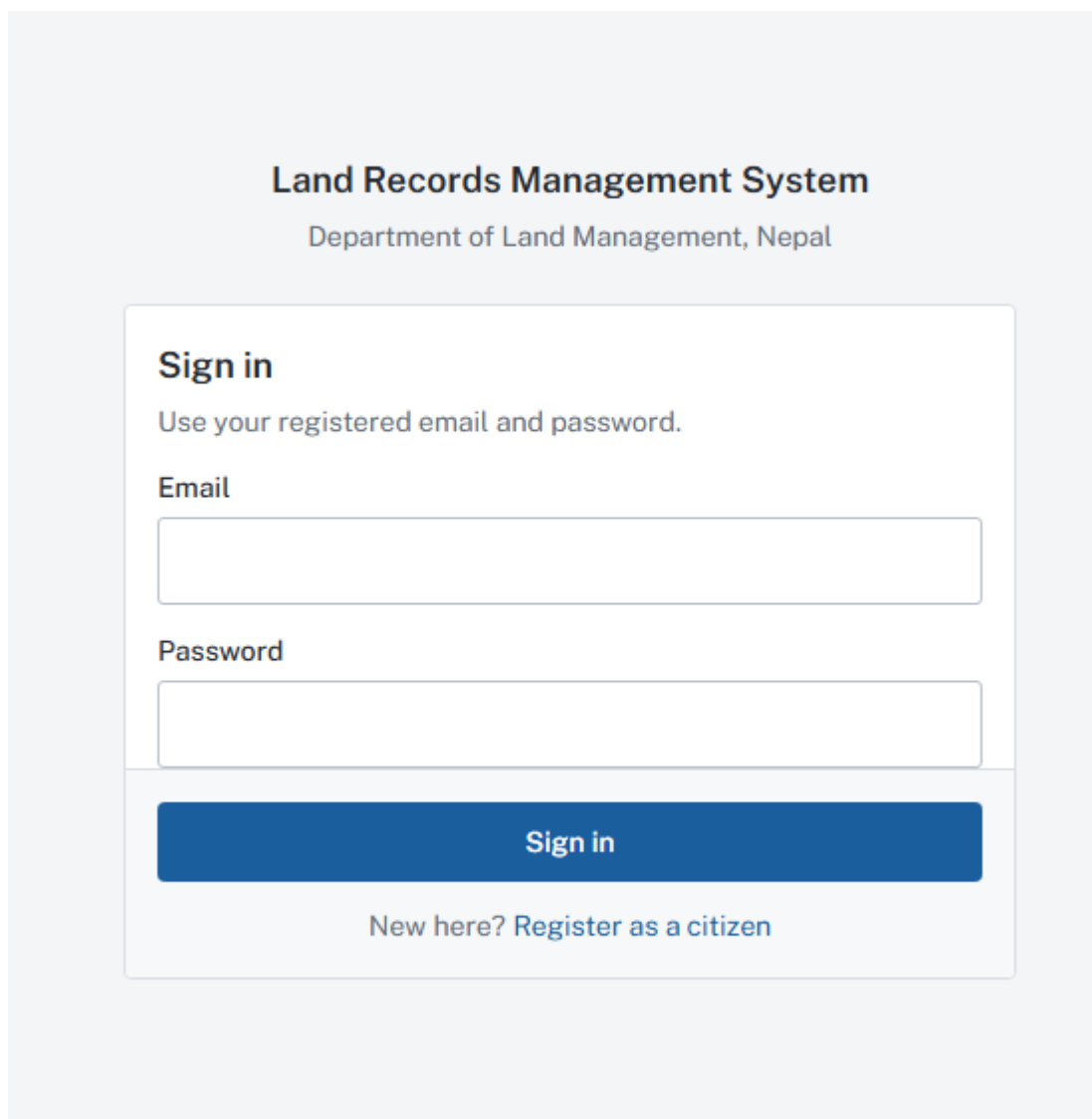
- [6] National Agency of Public Registry, "Georgia's blockchain implementation in land " 2024.

- [7] ChromaWay, "Sweden's Lantmäteriet land registry project," 2024.

- [8] Bitland, "Bitland project - Blockchain for rural land rights," 2024.

9. Appendices

Appendix A: Screenshot of the Web Application



The screenshot displays the login interface for the Land Records Management System. At the top, the system name is centered, followed by the department name. Below this, a 'Sign in' section contains instructions and input fields for email and password. A blue 'Sign in' button is positioned below the fields, and a link to register as a citizen is at the bottom of the form area.

Land Records Management System
Department of Land Management, Nepal

Sign in
Use your registered email and password.

Email

Password

Sign in

New here? [Register as a citizen](#)

Figure 15 Login Page

Land Records Management System

Department of Land Management, Nepal

Citizen registration

Your citizenship number is matched against land ownership records.

Full name

Email

Password (min. 6 characters)

Phone (optional)

Citizenship number

Register

Already registered? [Sign in](#)

Figure 16 Registration Page

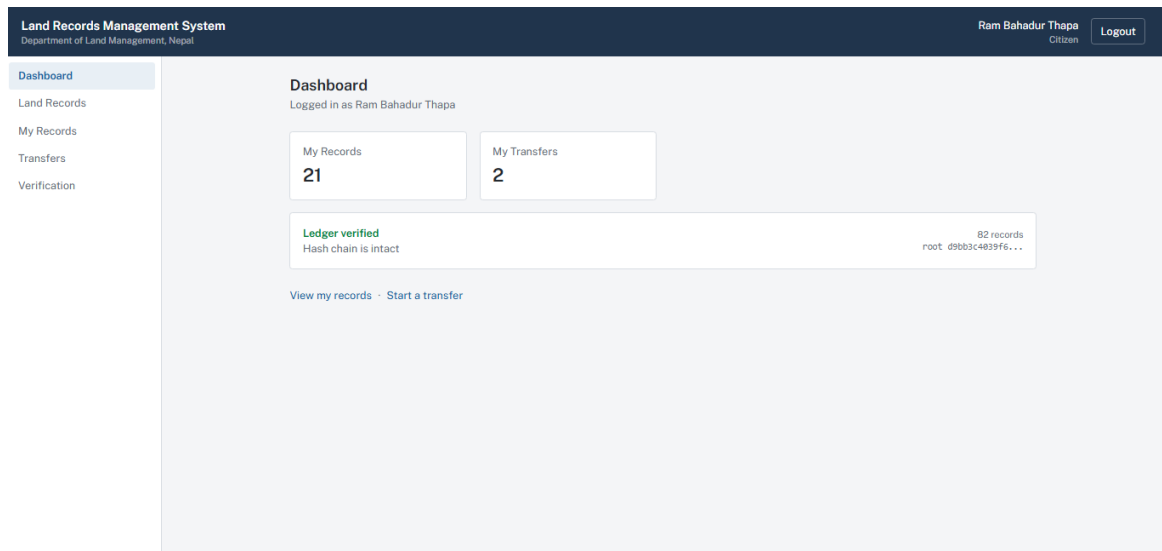


Figure 17 Citizen Interface

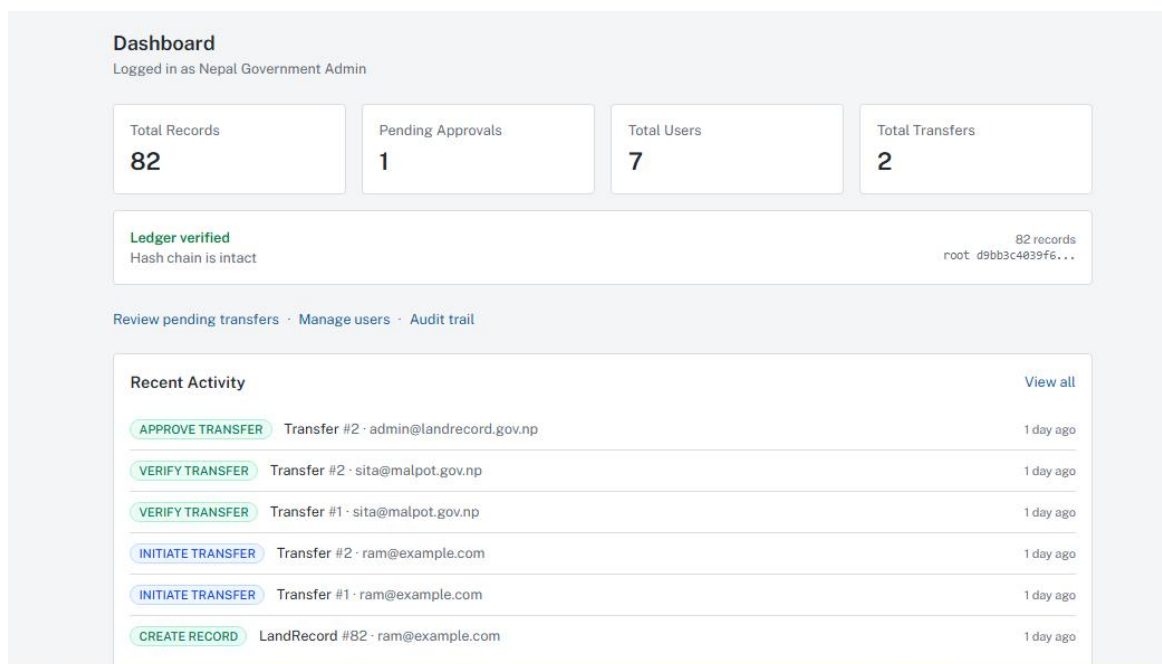


Figure 18 Admin Dashboard

[← Back to records](#)

Register Land Record

Land Details

Kitta Number

KTM-1001

Area (sq. meters)

500

District

Kathmandu

Municipality

Kathmandu Metropolitan City

Ward Number

10

Land Type

Select type ▾

Owner

Select owner ▾

Documents / Images (optional)

Choose Files No file chosen

Document Type

LALPURJA ▾

Register Land Record

Figure 19 Officer Dashboard

[← Back to records](#)

ktm - 100

Verify integrity

Record Details

Kitta Number	ktm - 100
Area	300 sq.m
District	Kathmandu
Municipality	KMC
Ward	1
Land Type	Aabad
Owner	Ram Bahadur Thapa
Created	7/1/2026

Hash Information

Record Hash

17f92c7663824c58b6f2221546be13aa5c873b17bee9897793ea8a57d97f83e8

Previous Record Hash

Genesis1s

Ownership History

- Ram Bahadur Thapa

7/1/2026 — Present

Documents (0)

Figure 20 View Records

[← Back to records](#)

ktm - 100

Verify integrity

Record verified

Record integrity verified

Computed: 17f92c7663024c58b6f2221546be13aa5c873b17bee9897793ea8a57d97f83e8

Stored: 17f92c7663024c58b6f2221546be13aa5c873b17bee9897793ea8a57d97f83e8

Root: d9bb3c4039f6bcaca8fc265443db9607f207c8bf1d901c7add4a8c4228cc94f7

Merkle Proof Path (7 steps):

Step 1: RIGHT → 78103ea751ab761d...

Step 2: RIGHT → 498843fa95deac1e...

Step 3: RIGHT → 5bcaf8fdf61654f1...

Step 4: RIGHT → 7b92b3269d3a2c50...

Step 5: RIGHT → de8bdf385075d867...

Step 6: RIGHT → 7d18fcc6dd931b4...

Step 7: RIGHT → 3be6e6f05409162a...

Record Details

Kitta Number	ktm - 100
Area	300 sq.m
District	Kathmandu
Municipality	KMC

Hash Information

Record Hash

17f92c7663024c58b6f2221546be13aa5c873b17bee9897793ea8a57d97f83e8

Previous Record Hash

Genesis

Figure 21 Verify Integrity

Appendix B: Screenshot of the Source Code

MerkleTreeEngine.java

```
1  package np.com.abhishekojha.landrecordmanagementbackend.merkle;
2
3  import java.nio.charset.StandardCharsets;
4  import java.security.MessageDigest;
5  import java.security.NoSuchAlgorithmException;
6  import java.util.ArrayList;
7  import java.util.List;
8
9  ✓ public class MerkleTreeEngine {
10
11      public static String sha256(String input) {
12          return sha256(input.getBytes(StandardCharsets.UTF_8));
13      }
14
15      // Hash raw bytes directly. Use this for binary files (PDFs, images) –
16      // decoding them into a String first corrupts the content before hashing.
17  ✓ public static String sha256(byte[] input) {
18      try {
19          MessageDigest digest = MessageDigest.getInstance("SHA-256");
20          byte[] hash = digest.digest(input);
21          StringBuilder hex = new StringBuilder(64);
22          for (byte b : hash) {
23              hex.append(String.format("%02x", b));
24          }
25          return hex.toString();
26      } catch (NoSuchAlgorithmException e) {
27          throw new RuntimeException("SHA-256 not available", e);
28      }
29  }
```

Figure 22 Source code of MerkleTree Engine

HashChainVerifier.java

```
import java.util.List;

public class HashChainVerifier {

    public static boolean verifyChain(List<LandRecordHashInput> records) {
        return findBrokenLink(records) == -1;
    }

    public static int findBrokenLink(List<LandRecordHashInput> records) {
        if (records == null || records.size() < 2) {
            return -1;
        }

        for (int i = 1; i < records.size(); i++) {
            String expectedHash = MerkleTreeEngine.computeRecordHash(records.get(i - 1));
            String actualPrevious = records.get(i).getPreviousOrGenesis();

            if (!expectedHash.equals(actualPrevious)) {
                return i;
            }
        }

        return -1;
    }
}
```

Figure 23 Source code of HashChain Verifier

landRecordapi.ts

```
1 import api from './axiosConfig';
2 import type { LandRecord, LandRecordRequest, OwnershipHistory } from '@types/landRecord';
3 import type { User } from '@types/user';
4 import type { PageResponse } from './auditApi';
5
6 export const landRecordApi = {
7   getAll: (search?: string, page: number = 0, size: number = 10) =>
8     api.get<PageResponse<LandRecord>>('/land-records', { params: search ? { search, page, size } : { page, size } }),
9
10  getById: (id: number) =>
11    api.get<LandRecord>(`/land-records/${id}`),
12
13  getHistory: (id: number) =>
14    api.get<OwnershipHistory[]>(`/land-records/${id}/history`),
15
16  getMyRecords: (page: number = 0, size: number = 10) =>
17    api.get<PageResponse<LandRecord>>('/citizen/my-records', { params: { page, size } }),
18
19  create: (data: LandRecordRequest) =>
20    api.post<LandRecord>('/officer/land-records', data),
21
22  createBulk: (data: LandRecordRequest[]) =>
23    api.post<LandRecord[]>('/officer/land-records/bulk', data),
24
25  searchBuyer: (citizenshipNumber: string, email: string) =>
26    api.get<User>('/citizen/buyer-search', { params: { citizenshipNumber, email } }),
27
28  getCitizensForOfficer: (page: number = 0, size: number = 10) =>
29    api.get<PageResponse<User>>('/officer/citizens', { params: { page, size } }),
30  };
```

Figure 24 SourceCode of LandRecord API

Log of Visits to Supervisor

Student Name: _____

Batch: _____

Roll No.: _____

Project/Internship Title: _____

Supervisor Name: _____

Campus: Amrit Campus, Thamel, Kathmandu

Department: Department of Computer Science & Information Technology

S.N.	Date	Discussion / Work Progress	Supervisor's Feedback / Suggestion	Student Signature	Supervisor Signature
1					
2					
3					
4					
5					

Supervisor Name: _____

Signature: _____

Date: _____